Search Write

Get unlimited access to the best of Medium for less than $1/week. Become a member

HANDS-ON TUTORIALS, INTUITIVE AUDIO DEEP LEARNING SERIES

Audio Deep Learning Made Simple: Sound Classification, Step-by-Step

An end-to-end example and architecture for audio deep learning's foundational application scenario, in plain English.

Ketan Doshi · Follow
Published in Towards Data Science · 12 min read · Mar 18, 2021

1.1K 21

Photo by bruce mars on Unsplash

Sound Classification is one of the most widely used applications in Audio Deep Learning. It involves learning to classify sounds and to predict the category of that sound. This type of problem can be applied to many practical scenarios e.g. classifying music clips to identify the genre of the music, or classifying short utterances by a set of speakers to identify the speaker based on the voice.

In this article, we will walk through a simple demo application so as to understand the approach used to solve such audio classification problems. My goal throughout will be to understand not just how something works but why it works that way.

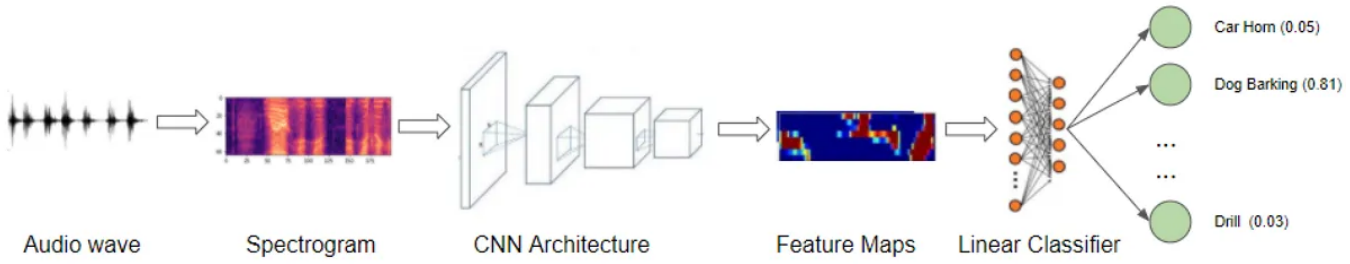
I have a few more articles in my audio deep learning series that you might find useful. They explore other fascinating topics in this space including how we prepare audio data for deep learning, why we use Mel Spectrograms for deep learning models and how they are generated and optimized.

1. State-of-the-Art Techniques *(What is sound and how it is digitized. What problems is audio deep learning solving in our daily lives. What are Spectrograms and why they are all-important.)*
2. Why Mel Spectrograms perform better *(Processing audio data in Python. What are Mel Spectrograms and how to generate them)*
3. Data Preparation and Augmentation *(Enhance Spectrograms features for optimal performance by hyper-parameter tuning and data augmentation)*
4. Automatic Speech Recognition *(Speech-to-Text algorithm and architecture, using CTC Loss and Decoding for aligning sequences.)*
5. Beam Search *(Algorithm commonly used by Speech-to-Text and NLP applications to enhance predictions)*

Audio Classification

Just like classifying hand-written digits using the MNIST dataset is considered a ‘Hello World”-type problem for Computer Vision, we can think of this application as the introductory problem for audio deep learning.

We will start with sound files, convert them into spectrograms, input them into a CNN plus Linear Classifier model, and produce predictions about the class to which the sound belongs.



Audio Classification application (Image by Author)

There are many suitable datasets available for sounds of different types. These datasets contain a large number of audio samples, along with a class label for each sample that identifies what type of sound it is, based on the problem you are trying to address.

These class labels can often be obtained from some part of the filename of the audio sample or from the sub-folder name in which the file is located. Alternately the class labels are specified in a separate metadata file, usually in TXT, JSON, or CSV format.

Example problem — Classifying ordinary city sounds

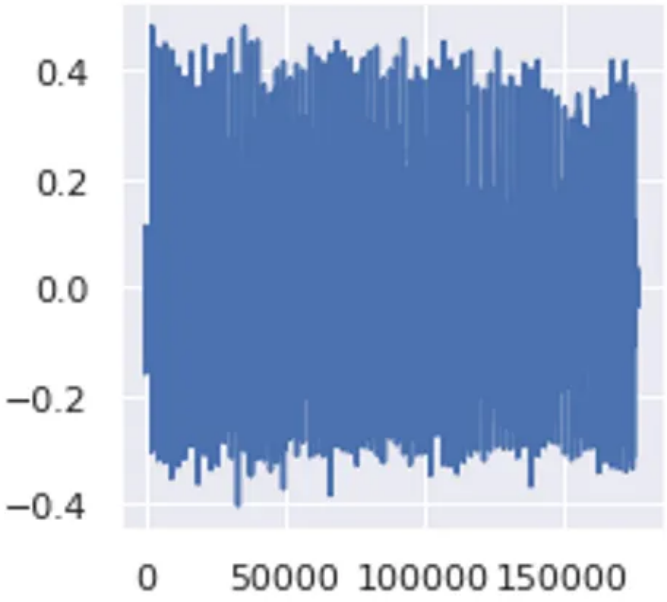
For our demo, we will use the Urban Sound 8K dataset that consists of a corpus of ordinary sounds recorded from day-to-day city life. The sounds are taken from 10 classes such as drilling, dogs barking, and sirens. Each sound sample is labeled with the class to which it belongs.

After downloading the dataset, we see that it consists of two parts:

- **Audio files** in the '*audio*' folder: It has 10 sub-folders named '*fold1*' through '*fold10*'. Each sub-folder contains a number of '*.wav*' audio samples eg. '*fold1/103074–7–1–0.wav*'
- **Metadata** in the 'metadata' folder: It has a file '*UrbanSound8K.csv*' that contains information about each audio sample in the dataset such as its filename, its class label, the 'fold' sub-folder location, and so on. The class label is a numeric Class ID from 0–9 for each of the 10 classes. eg. the number 0 means air conditioner, 1 is a car horn, and so on.

The samples are around 4 seconds in length. Here's what one sample looks

like:



An audio sample of a drill (Image by Author)

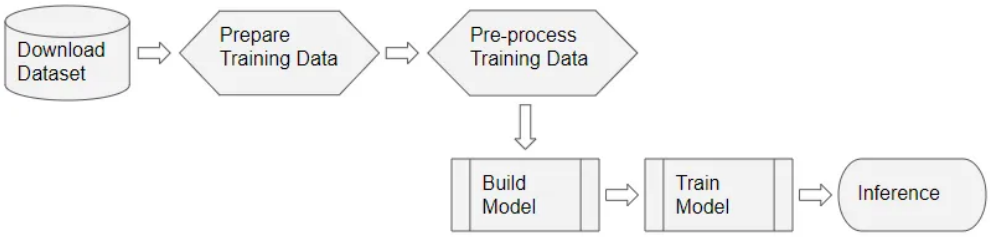
```
Sample rate = 44100, Channels = 2, Bits = 16, Encoding type = PCM_S
```

Sample Rate, Number of Channels, Bits, and Audio Encoding

The recommendation of the dataset creators is to use the folds for doing 10-fold cross-validation to report metrics and evaluate the performance of your model. However, since our goal in this article is primarily as a demo of an audio deep learning example rather than to obtain the best metrics, we will ignore the folds and treat all the samples simply as one large dataset.

Prepare training data

As for most deep learning problems, we will follow these steps:



Deep Learning Workflow (Image by Author)

The training data for this problem will be fairly simple:

- The features (X) are the audio file paths
- The target labels (y) are the class names

Since the dataset has a metadata file that contains this information already, we can use that directly. The metadata contains information about each audio file.

	slice_file_name	fsID	start	end	salience	fold	classID	class
0	100032-3-0-0.wav	100032	0.0	0.317551	1	5	3	dog_bark
1	100263-2-0-117.wav	100263	58.5	62.500000	1	5	2	children_playing
2	100263-2-0-121.wav	100263	60.5	64.500000	1	5	2	children_playing
3	100263-2-0-126.wav	100263	63.0	67.000000	1	5	2	children_playing
4	100263-2-0-137.wav	100263	68.5	72.500000	1	5	2	children_playing

Since it is a CSV file, we can use Pandas to read it. We can prepare the feature and label data from the metadata.

```
# ----------------------------
# Prepare training data from Metadata file
# ----------------------------
import pandas as pd
from pathlib import Path

download_path = Path.cwd()/'UrbanSound8K'

# Read metadata file
metadata_file = download_path/'metadata'/'UrbanSound8K.csv'
df = pd.read_csv(metadata_file)
df.head()

# Construct file path by concatenating fold and file name
df['relative_path'] = '/fold' + df['fold'].astype(str) + '/' + df['slice_file_name'].a

# Take relevant columns
df = df[['relative_path', 'classID']]
df.head()
```

audio_metadata.py hosted with ♥ by GitHub view raw

This gives us the information we need for our training data.

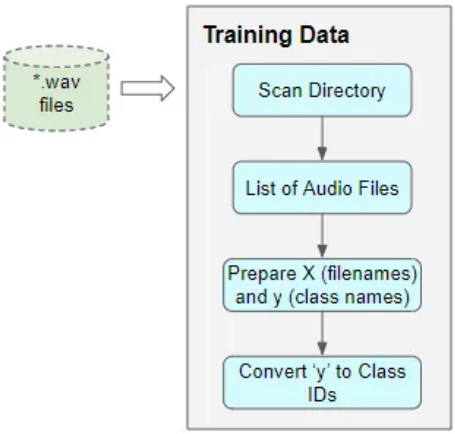
	relative_path	classID
0	/fold5/100032-3-0-0.wav	3
1	/fold5/100263-2-0-117.wav	2
2	/fold5/100263-2-0-121.wav	2
3	/fold5/100263-2-0-126.wav	2
4	/fold5/100263-2-0-137.wav	2

Training data with audio file paths and class IDs

Scan the audio file directory when metadata isn't available

Having the metadata file made things easy for us. How would we prepare our data for datasets that do not contain a metadata file?

Many datasets consist of only audio files arranged in a folder structure from which class labels can be derived. To prepare our training data in this format, we would do the following:



Preparing Training Data when metadata isn't available (Image by Author)

- Scan the directory and prepare a list of all the audio file paths.
- Extract the class label from each file name, or from the name of the parent sub-folder
- Map each class name from text to a numeric class ID

With or without metadata, the result would be the same — features consisting of a list of audio file names and target labels consisting of class IDs.

Audio Pre-processing: Define Transforms

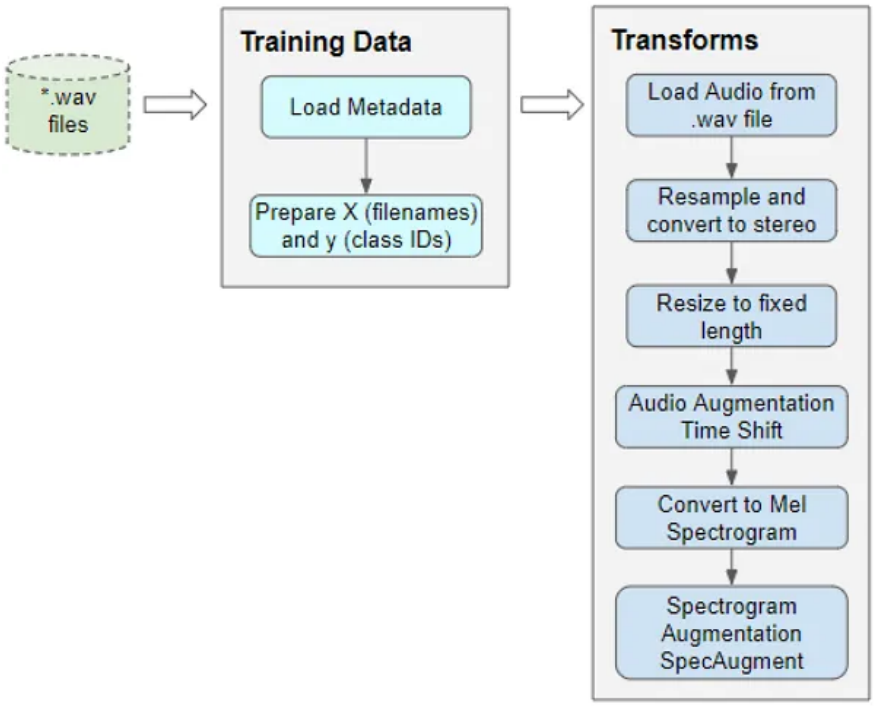
This training data with audio file paths cannot be input directly into the model. We have to load the audio data from the file and process it so that it is in a format that the model expects.

This audio pre-processing will all be done dynamically at runtime when we will read and load the audio files. This approach is similar to what we would do with image files as well. Since audio data, like image data, can be fairly large and memory-intensive, we don't want to read the entire dataset into memory all at once, ahead of time. So we keep only the audio file names (or image file names) in our training data.

Then, at runtime, as we train the model one batch at a time, we will load the audio data for that batch and process it by applying a series of transforms to the audio. That way we keep audio data for only one batch in memory at a time.

With image data, we might have a pipeline of transforms where we first read the image file as pixels and load it. Then we might apply some image processing steps to reshape and resize the data, crop them to a fixed size and convert them into grayscale from RGB. We might also apply some image augmentation steps like rotation, flips, and so on.

The processing for audio data is very similar. Right now we're only defining the functions, they will be run a little later when we feed data to the model during training.



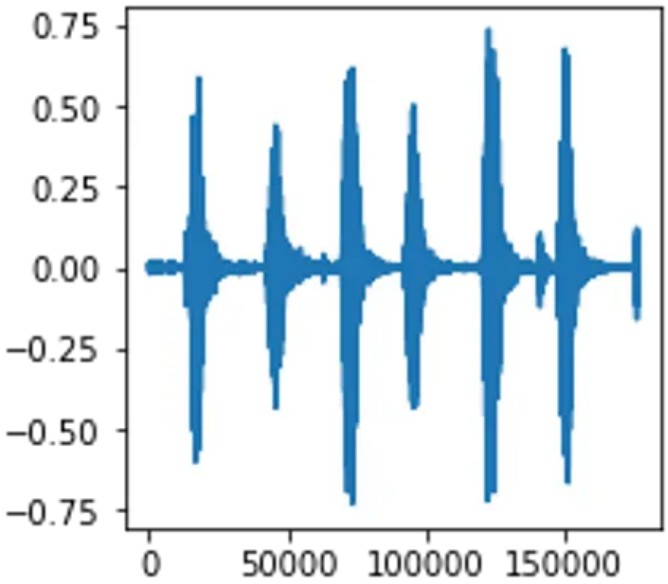
Pre-processing the training data for input to our model (Image by Author)

Read audio from a file

The first thing we need is to read and load the audio file in ".wav" format. Since we are using Pytorch for this example, the implementation below uses torchaudio for the audio processing, but librosa will work just as well.

```
import math, random
import torch
import torchaudio
from torchaudio import transforms
from IPython.display import Audio

class AudioUtil():
  # ----------------------------
  # Load an audio file. Return the signal as a tensor and the sample rate
  # ----------------------------
  @staticmethod
  def open(audio_file):
    sig, sr = torchaudio.load(audio_file)
    return (sig, sr)
```

audio_util_load.py hosted with ♥ by GitHub view raw



Audio wave loaded from a file (Image by Author)

Convert to two channels

Some of the sound files are mono (ie. 1 audio channel) while most of them are stereo (ie. 2 audio channels). Since our model expects all items to have the same dimensions, we will convert the mono files to stereo, by duplicating the first channel to the second.

```
# ----------------------------
# Convert the given audio to the desired number of channels
# ----------------------------
@staticmethod
def rechannel(aud, new_channel):
  sig, sr = aud

  if (sig.shape[0] == new_channel):
    # Nothing to do
    return aud

  if (new_channel == 1):
    # Convert from stereo to mono by selecting only the first channel
    resig = sig[:1, :]
  else:
    # Convert from mono to stereo by duplicating the first channel
    resig = torch.cat([sig, sig])

  return ((resig, sr))
```

transform_rechannel.py hosted with ❤ by GitHub view raw

Standardize sampling rate

Some of the sound files are sampled at a sample rate of 48000Hz, while most are sampled at a rate of 44100Hz. This means that 1 second of audio will have an array size of 48000 for some sound files, while it will have a smaller array size of 44100 for the others. Once again, we must standardize and convert all audio to the same sampling rate so that all arrays have the same dimensions.

```
# ----------------------------
# Since Resample applies to a single channel, we resample one channel at a time
# ----------------------------
@staticmethod
def resample(aud, newsr):
  sig, sr = aud

  if (sr == newsr):
    # Nothing to do
    return aud

  num_channels = sig.shape[0]
  # Resample first channel
  resig = torchaudio.transforms.Resample(sr, newsr)(sig[:1,:])
  if (num_channels > 1):
    # Resample the second channel and merge both channels
    retwo = torchaudio.transforms.Resample(sr, newsr)(sig[1:,:])
    resig = torch.cat([resig, retwo])

  return ((resig, newsr))
```

transform_resample.py hosted with ♥ by GitHub view raw

Resize to the same length

We then resize all the audio samples to have the same length by either extending its duration by padding it with silence, or by truncating it. We add that method to our AudioUtil class.

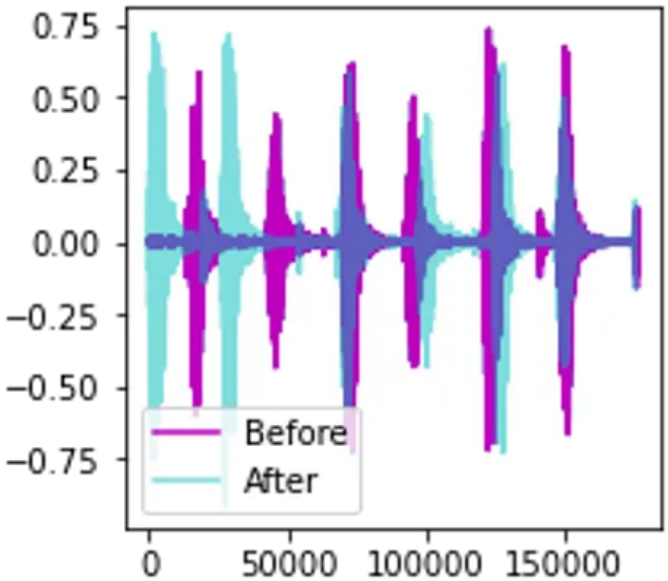
```
# ----------------------------
# Pad (or truncate) the signal to a fixed length 'max_ms' in milliseconds
# ----------------------------
@staticmethod
def pad_trunc(aud, max_ms):
  sig, sr = aud
  num_rows, sig_len = sig.shape
  max_len = sr//1000 * max_ms

  if (sig_len > max_len):
    # Truncate the signal to the given length
    sig = sig[:,:max_len]

  elif (sig_len < max_len):
    # Length of padding to add at the beginning and end of the signal
    pad_begin_len = random.randint(0, max_len - sig_len)
```

```
      pad_end_len = max_len - sig_len - pad_begin_len

      # Pad with 0s
      pad_begin = torch.zeros((num_rows, pad_begin_len))
      pad_end = torch.zeros((num_rows, pad_end_len))

      sig = torch.cat((pad_begin, sig, pad_end), 1)

    return (sig, sr)
```

transform_padtrunc.py hosted with ♥ by GitHub view raw

Data Augmentation: Time Shift

Next, we can do data augmentation on the raw audio signal by applying a Time Shift to shift the audio to the left or the right by a random amount. I go into a lot more detail about this and other data augmentation techniques in this article.



Time Shift of the audio wave (Image by Author)

```
  # ----------------------------
  # Shifts the signal to the left or right by some percent. Values at the end
  # are 'wrapped around' to the start of the transformed signal.
  # ----------------------------
  @staticmethod
  def time_shift(aud, shift_limit):
    sig,sr = aud
    _, sig_len = sig.shape
    shift_amt = int(random.random() * shift_limit * sig_len)
    return (sig.roll(shift_amt), sr)
```

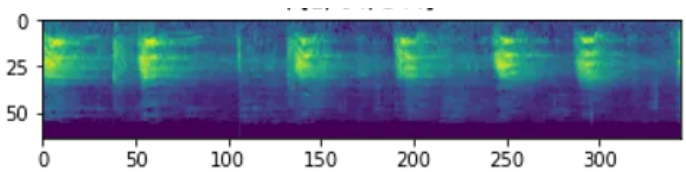
transform_timeshift.py hosted with ♥ by GitHub view raw

Mel Spectrogram

We then convert the augmented audio to a Mel Spectrogram. They capture the essential features of the audio and are often the most suitable way to input audio data into deep learning models. To get more background about this, you might want to read my articles (here and here) which explain in simple words what a Mel Spectrogram is, why they are crucial for audio deep learning, as well as how they are generated and how to tune them for getting the best performance from your models.

```
# ----------------------------
# Generate a Spectrogram
# ----------------------------
@staticmethod
def spectro_gram(aud, n_mels=64, n_fft=1024, hop_len=None):
  sig,sr = aud
  top_db = 80

  # spec has shape [channel, n_mels, time], where channel is mono, stereo etc
  spec = transforms.MelSpectrogram(sr, n_fft=n_fft, hop_length=hop_len, n_mels=n_mel

  # Convert to decibels
  spec = transforms.AmplitudeToDB(top_db=top_db)(spec)
  return (spec)
```

transform_mel.py hosted with ♥ by GitHub view raw



Mel Spectrogram of an audio wave (Image by Author)

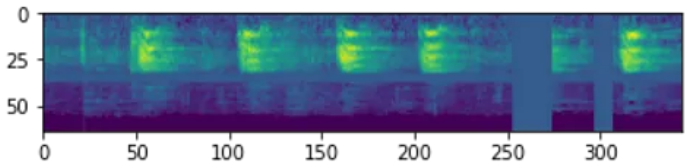
Data Augmentation: Time and Frequency Masking

Now we can do another round of augmentation, this time on the Mel Spectrogram rather than on the raw audio. We will use a technique called SpecAugment that uses these two methods:

- Frequency mask — randomly mask out a range of consecutive frequencies by adding horizontal bars on the spectrogram.
- Time mask — similar to frequency masks, except that we randomly block out ranges of time from the spectrogram by using vertical bars.

```
# ----------------------------
# Augment the Spectrogram by masking out some sections of it in both the frequency
# dimension (ie. horizontal bars) and the time dimension (vertical bars) to prevent
# overfitting and to help the model generalise better. The masked sections are
# replaced with the mean value.
# ----------------------------
@staticmethod
def spectro_augment(spec, max_mask_pct=0.1, n_freq_masks=1, n_time_masks=1):
  _, n_mels, n_steps = spec.shape
  mask_value = spec.mean()
  aug_spec = spec

  freq_mask_param = max_mask_pct * n_mels
  for _ in range(n_freq_masks):
    aug_spec = transforms.FrequencyMasking(freq_mask_param)(aug_spec, mask_value)

  time_mask_param = max_mask_pct * n_steps
  for _ in range(n_time_masks):
    aug_spec = transforms.TimeMasking(time_mask_param)(aug_spec, mask_value)

  return aug_spec
```

transform_spec_augment.py hosted with ♥ by GitHub view raw



Mel Spectrogram after SpecAugment. Notice the horizontal and vertical mask bands (Image by Author)

Define Custom Data Loader

Now that we have defined all the pre-processing transform functions we will define a custom Pytorch Dataset object.

To feed your data to a model with Pytorch, we need two objects:

- A custom Dataset object that uses all the audio transforms to pre-process an audio file and prepares one data item at a time.
- A built-in DataLoader object that uses the Dataset object to fetch individual data items and packages them into a batch of data.

```
from torch.utils.data import DataLoader, Dataset, random_split
import torchaudio

# ----------------------------
# Sound Dataset
```

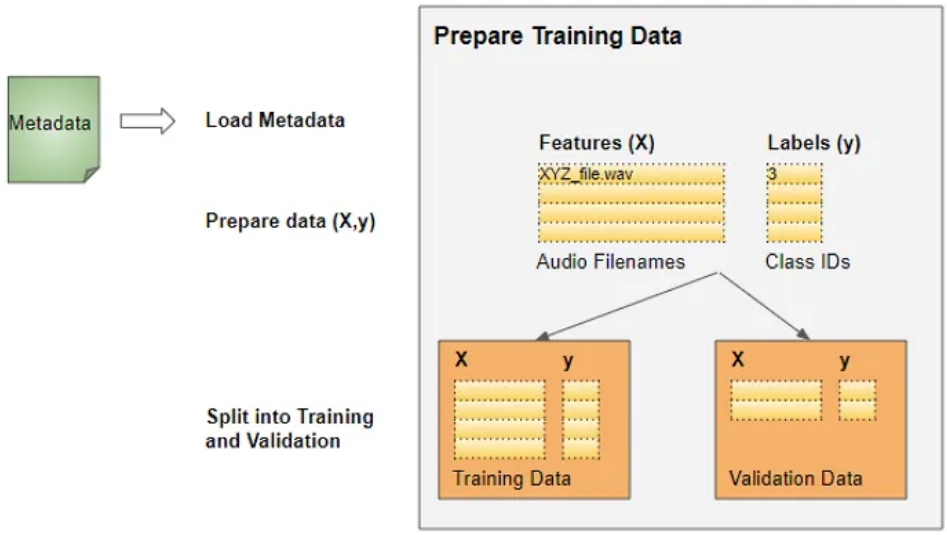
```python
# ----------------------------
class SoundDS(Dataset):
  def __init__(self, df, data_path):
    self.df = df
    self.data_path = str(data_path)
    self.duration = 4000
    self.sr = 44100
    self.channel = 2
    self.shift_pct = 0.4

  # ----------------------------
  # Number of items in dataset
  # ----------------------------
  def __len__(self):
    return len(self.df)

  # ----------------------------
  # Get i'th item in dataset
  # ----------------------------
  def __getitem__(self, idx):
    # Absolute file path of the audio file - concatenate the audio directory with
    # the relative path
    audio_file = self.data_path + self.df.loc[idx, 'relative_path']
    # Get the Class ID
    class_id = self.df.loc[idx, 'classID']

    aud = AudioUtil.open(audio_file)
    # Some sounds have a higher sample rate, or fewer channels compared to the
    # majority. So make all sounds have the same number of channels and same
    # sample rate. Unless the sample rate is the same, the pad_trunc will still
    # result in arrays of different lengths, even though the sound duration is
    # the same.
    reaud = AudioUtil.resample(aud, self.sr)
    rechan = AudioUtil.rechannel(reaud, self.channel)

    dur_aud = AudioUtil.pad_trunc(rechan, self.duration)
    shift_aud = AudioUtil.time_shift(dur_aud, self.shift_pct)
    sgram = AudioUtil.spectro_gram(shift_aud, n_mels=64, n_fft=1024, hop_len=None)
    aug_sgram = AudioUtil.spectro_augment(sgram, max_mask_pct=0.1, n_freq_masks=2, n_t

    return aug_sgram, class_id
```

sound_classification_dataset.py hosted with ❤ by GitHub view raw

Prepare Batches of Data with the Data Loader

All of the functions we need to input our data to the model have now been defined.

We use our custom Dataset to load the Features and Labels from our Pandas dataframe and split that data randomly in an 80:20 ratio into training and validation sets. We then use them to create our training and validation Data Loaders.

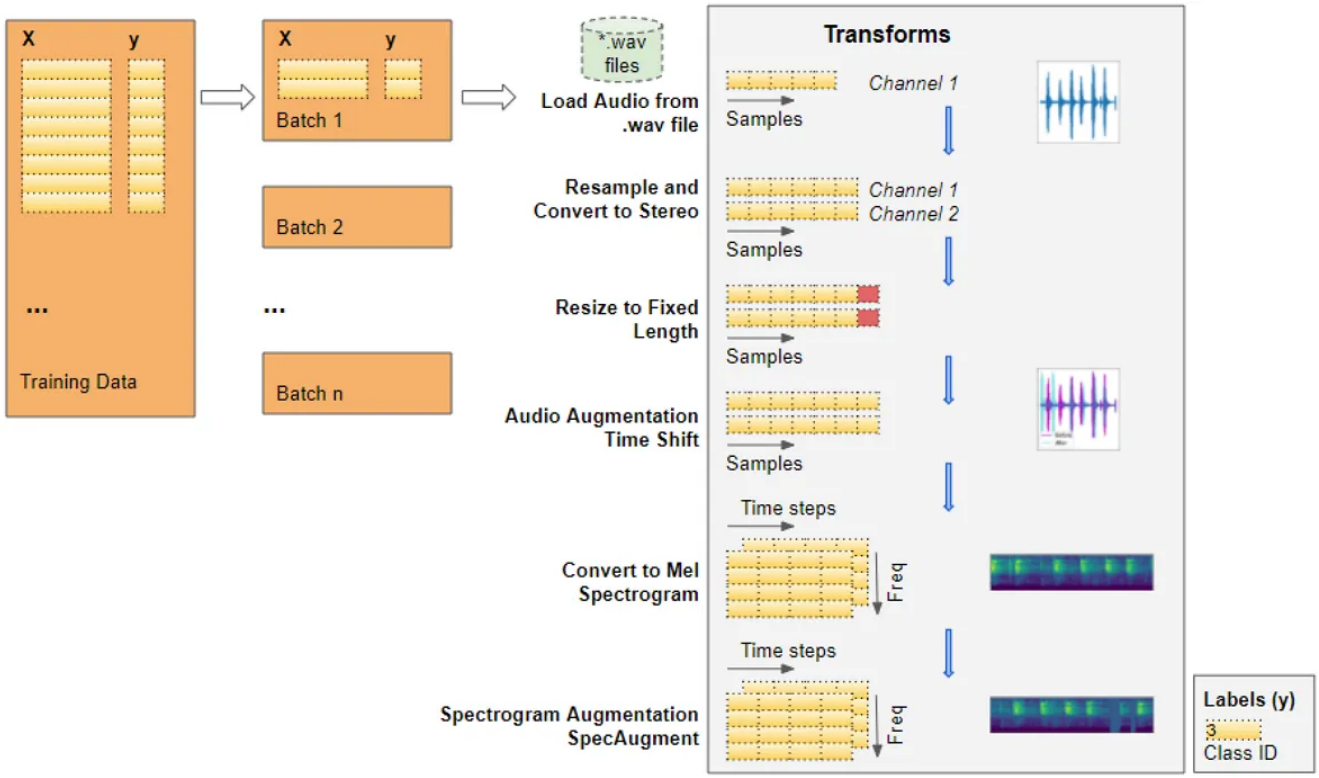


Split our data for training and validation (Image by Author)

```
from torch.utils.data import random_split

myds = SoundDS(df, data_path)

# Random split of 80:20 between training and validation
num_items = len(myds)
num_train = round(num_items * 0.8)
num_val = num_items - num_train
train_ds, val_ds = random_split(myds, [num_train, num_val])

# Create training and validation data loaders
train_dl = torch.utils.data.DataLoader(train_ds, batch_size=16, shuffle=True)
val_dl = torch.utils.data.DataLoader(val_ds, batch_size=16, shuffle=False)
```

sound_classification_split.py hosted with ❤ by GitHub view raw

When we start training, the Data Loader will randomly fetch one batch of input Features containing the list of audio file names and run the pre-processing audio transforms on each audio file. It will also fetch a batch of the corresponding target Labels containing the class IDs. Thus it will output one batch of training data at a time, which can directly be fed as input to our deep learning model.



Data Loader applies transforms and prepares one batch of data at a time (Image by Author)

Let's walk through the steps as our data gets transformed, starting with an audio file:

- The audio from the file gets loaded into a Numpy array of shape (num_channels, num_samples). Most of the audio is sampled at 44.1kHz and is about 4 seconds in duration, resulting in 44,100 * 4 = 176,400 samples. If the audio has 1 channel, the shape of the array will be (1, 176,400). Similarly, audio of 4 seconds duration with 2 channels and sampled at 48kHz will have 192,000 samples and a shape of (2, 192,000).
- Since the channels and sampling rates of each audio are different, the next two transforms resample the audio to a standard 44.1kHz and to a standard 2 channels.
- Since some audio clips might be more or less than 4 seconds, we also standardize the audio duration to a fixed length of 4 seconds. Now arrays for all items have the same shape of (2, 176,400)
- The Time Shift data augmentation now randomly shifts each audio

sample forward or backward. The shapes are unchanged.

- The augmented audio is now converted into a Mel Spectrogram, resulting in a shape of (num_channels, Mel freq_bands, time_steps) = (2, 64, 344)
- The SpecAugment data augmentation now randomly applies Time and Frequency Masks to the Mel Spectrograms. The shapes are unchanged.

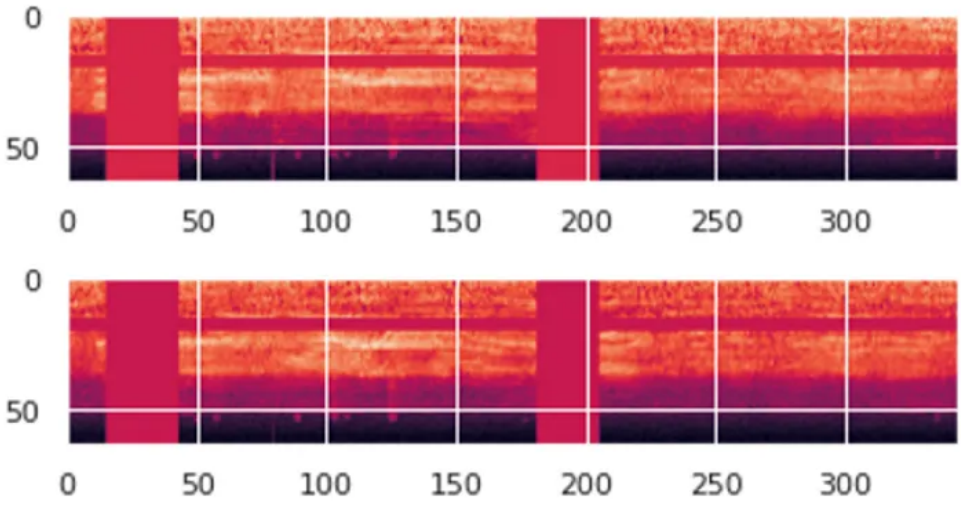
Thus, each batch will have two tensors, one for the *X* feature data containing the Mel Spectrograms and the other for the *y* target labels containing numeric Class IDs. The batches are picked randomly from the training data for each training epoch.

Each batch has a shape of (batch_sz, num_channels, Mel freq_bands, time_steps)

```
(torch.Size([16, 2, 64, 344]), torch.Size([16]))
```

A batch of (X, y) data

We can visualize one item from the batch. We see the Mel Spectrogram with vertical and horizontal stripes showing the Frequency and Time Masking data augmentation.

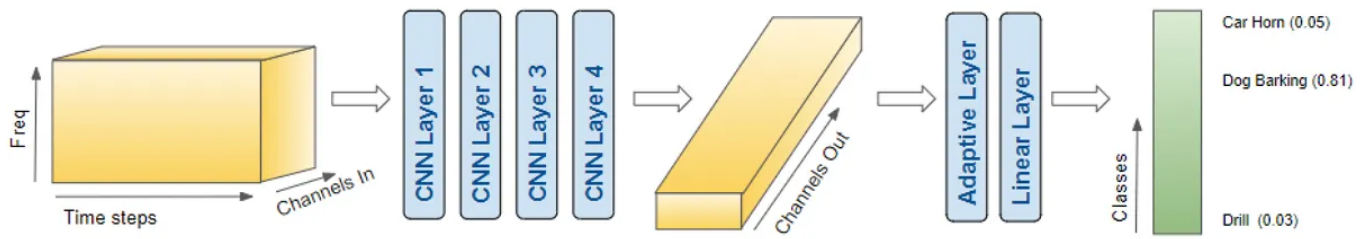


The data is now ready for input to the model.

Create Model

The data processing steps that we just did are the most unique aspects of our audio classification problem. From here on, the model and training procedure are quite similar to what is commonly used in a standard image classification problem and are not specific to audio deep learning.

Since our data now consists of Spectrogram images, we build a CNN classification architecture to process them. It has four convolutional blocks which generate the feature maps. That data is then reshaped into the format we need so it can be input into the linear classifier layer, which finally outputs the predictions for the 10 classes.



The model takes a batch of pre-processed data and outputs class predictions (Image by Author)

A few more details about how the model processes a batch of data:

- A batch of images is input to the model with shape (batch_sz, num_channels, Mel freq_bands, time_steps) ie. (16, 2, 64, 344).
- Each CNN layer applies its filters to step up the image depth ie. number of channels. The image width and height are reduced as the kernels and strides are applied. Finally, after passing through the four CNN layers, we get the output feature maps ie. (16, 64, 4, 22).
- This gets pooled and flattened to a shape of (16, 64) and then input to the Linear layer.
- The Linear layer outputs one prediction score per class ie. (16, 10)

Training

We are now ready to create the training loop to train the model.

We define the functions for the optimizer, loss, and scheduler to dynamically vary our learning rate as training progresses, which usually allows training to converge in fewer epochs.

We train the model for several epochs, processing a batch of data in each iteration. We keep track of a simple accuracy metric which measures the percentage of correct predictions.

Inference

Ordinarily, as part of the training loop, we would also evaluate our metrics on the validation data. We would then do inference on unseen data, perhaps by keeping aside a test dataset from the original data. However, for the purposes of this demo, we will use the validation data for this purpose.

We run an inference loop taking care to disable the gradient updates. The forward pass is executed with the model to get predictions, but we do not need to backpropagate or run the optimizer.

Conclusion

We have now seen an end-to-end example of sound classification which is one of the most foundational problems in audio deep learning. Not only is this used in a wide range of applications, but many of the concepts and techniques that we covered here will be relevant to more complicated audio problems such as automatic speech recognition where we start with human speech, understand what people are saying, and convert it to text.

And finally, if you liked this article, you might also enjoy my other series on Transformers, Geolocation Machine Learning, and Image Caption architectures.

Transformers Explained Visually (Part 1): Overview of Functionality

A Gentle Guide to Transformers for NLP, and why they are better than RNNs, in Plain English. How Attention helps...

towardsdatascience.com

Leveraging Geolocation Data for Machine Learning: Essential Techniques

A Gentle Guide to Feature Engineering and Visualization with Geospatial data, in Plain English

towardsdatascience.com

Image Captions with Deep Learning: State-of-the-Art Architectures

A Gentle Guide to Image Feature Encoders, Sequence Decoders, Attention, and Multi-modal Architectures, in Plain English

towardsdatascience.com

Let’s keep learning!

Artificial Intelligence Data Science Speech Recognition Editors Pick Hands On Tutorials

Written by Ketan Doshi

Follow

4.8K Followers · Writer for Towards Data Science

Machine Learning and Big Data

More from Ketan Doshi and Towards Data Science

Ketan Doshi in Towards Data Science

Transformers Explained Visually (Part 3): Multi-head Attention,

A Gentle Guide to the inner workings of Self-Attention, Encoder-Decoder Attention,

11 min read · Jan 17, 2021

2.3K 25

Adrian H. Raudaschl in Towards Data Science

Forget RAG, the Future is RAG-Fusion

The Next Frontier of Search: Retrieval Augmented Generation meets Reciprocal

· 10 min read · Oct 6

2.3K 24

Marco Peixeiro in Towards Data Science

TimeGPT: The First Foundation Model for Time Series Forecasting

Explore the first generative pre-trained forecasting model and apply it in a project

· 12 min read · Oct 24

1.97K 18

Ketan Doshi in Towards Data Science

Transformers Explained Visually (Part 1): Overview of Functionality

A Gentle Guide to Transformers for NLP, and why they are better than RNNs, in Plain

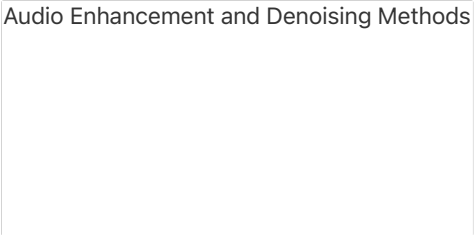
10 min read · Dec 13, 2020

2.9K 15

See all from Ketan Doshi

See all from Towards Data Science

Recommended from Medium



Ankur Dhuriya

Audio Enhancement and Denoising Methods

Explore a range of powerful methods and techniques for audio enhancement and

6 min read · Jul 11

20

Leonie Monigatti in Towards Data Science

Audio Classification with Deep Learning in Python

Fine-tuning image models to tackle domain shift and class imbalance with PyTorch and

· 11 min read · Apr 4

415 6

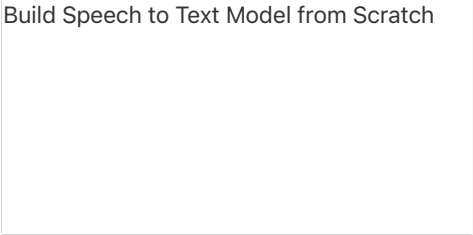
Lists

Predictive Modeling w/ Python
20 stories · 568 saves

ChatGPT prompts
27 stories · 621 saves

ChatGPT
22 stories · 245 saves

AI Regulation
6 stories · 178 saves



Zahra Ahmad in MLearning.ai

Build Speech to Text Model from Scratch

What we need to train a model that converts speech into text from collecting data to

· 4 min read · Jun 1

71

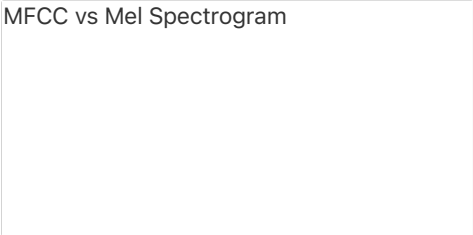
David Oluyale

Audio Classification using Deep Learning and TensorFlow: A Step-

Audio classification is a fascinating field with numerous real-world applications, from

5 min read · Sep 26

28



Tiya Vaj

MFCC vs Mel Spectrogram

MFCC (Mel-Frequency Cepstral Coefficients) and Mel Spectrogram do not generate the

1 min read · Sep 6

La Javaness R&D

Speaker Diarization: An Introductory Overview

Speaker diarization is the process of automatically identifying and segmenting an

7 min read · Jul 17

177

See more recommendations

Help Status About Careers Blog Privacy Terms Text to speech Teams